

CONCEPT PHASE:

In this phase, I have created a comprehensive database schema for an Airbnb platform that connects host and guest for short-term accommodations. The primary goal of database is to store and manage the information related to users (hosts, guests and admins), properties, booking, payments, reviews, and other relevant data in efficiently and well structured.

Lots of challenges are created because of robust, unstandardized data structure of platform's functionalities such as user profiles, property listings, booking transaction, reviews which tend to data inconsistencies, partial updates, lost updates and hardware issue or concurrent user access.

SOLUTION APPROACHES:

1. Requirements Specification:

- Developed and defined the requirement specification document outlining necessary functions associated with each user roles.

2. Entity-Relationship Model (ERM):

- Developed an Entity-Relationship Model (ERM) helps to visualize the relationship between different entities related on works. Using the consistent notation (e.g. Chen notation) specify cardinality helps to display the data flow and relationship to understand the dependency on each entities. Creating a data dictionary helps to describe attributes and their data types of each table.

Requirements Specification:

Roles (User Groups):

1. **Host:** A user who lists accommodations.
2. **Guest:** A user who books accommodations.
3. **Admin:** A platform administrator who manages users, listings, and transactions.

Actions Performed by Roles:

- **Host:** Users who create and manage their properties (apartments, rooms, etc.) for rent, upload photos and description of properties, set pricing and availability, communicate with guests, receive payments after guest check-in and rate guests after their stay.
- **Guest:** Users who search for properties based on location, price, and amenities, book properties, pay for booking via the platform, communication with hosts and rate hosts properties after their stay.

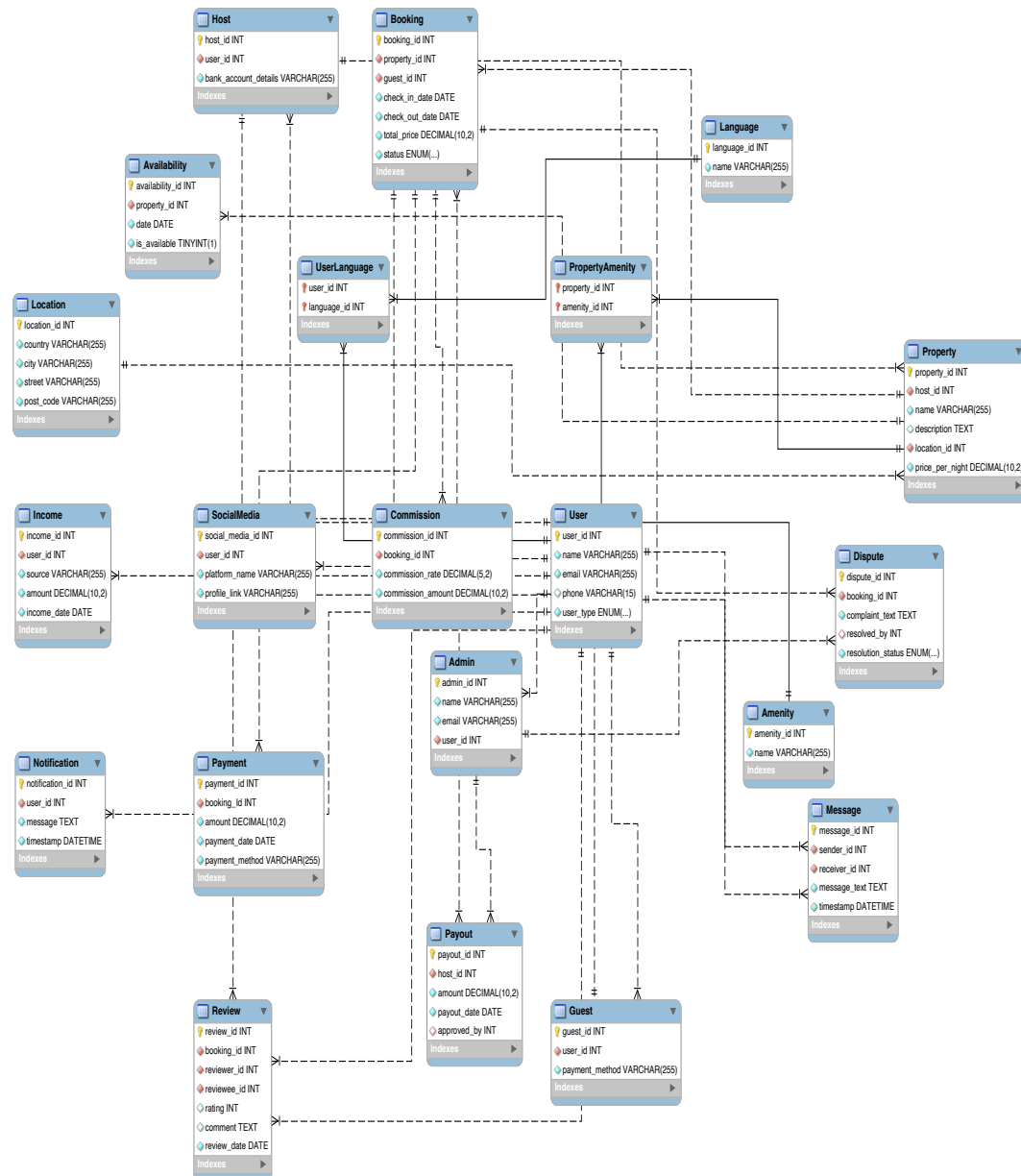
- **Admin:** Platform administrators who monitor and manage the user account, handle disputes between hosts and guests, ensure compliance with platform policies, and manage payments and commissions.
- **System:** Ensuring the database on well-structured and logical tables help to eliminate the redundancy and maintain the data integrity, update and reminder the user booking.
- **Overall Actions:** On this database, user can select the different language to search their dreams holiday apartment or room to spend their quality of time looking the image and details of amenities of property.

Required Data and Function:

- **User Functions:** Mention all the profile information for hosts and guests (e.g., name email, phone number, social media links).
- **Guest Functions:** Searching for propertied based on location, price and amenities, book a property and rate host and properties after the stay.
- **Host Functions:** Create and manage property listings, upload photos and descriptions of the property, set pricing and availability.
- **Admin Functions:** Manage user accounts (hosts and guests), resolve disputes between hosts and guests, and monitor transactions and ensure platform policies are followed.
- **System Functions:** Remainder message, maintain data integrity, eliminate the redundancy, social media management, multilanguage support.
- **Overall Functions:**
 - ⇒ **Message Functionality:** Hosts and guests should be able to communicate through the platform.
 - ⇒ **Multilanguage support:** Users prefer language and support.
 - ⇒ **Social media Functionality:** Details of property and communication support.
 - ⇒ **Browsing Stream:** Search and filter based on user need.
 - ⇒ **Reservation Functionality:** Pre-book accepted, provide check-in and check-out optionality on site, secure payment method.
 - ⇒ **Review Functionality:** Both hosts and guests' rate and review each other.
 - ⇒ **Income Report:** Access yearly income, source, amount and income date.
 - ⇒ **Notification Functionality:** Generate notification for user based.
 - ⇒ **Commission Functionality:** Commission offer on scheme or campaign.
 - ⇒ **Location Functionality:** Various location availability for customers preference.
 - ⇒ **Dispute Functionality:** Management system handle dispute rise by guests and hosts.
 - ⇒ **Security Management:** Secure web browsing, data protection and log security-related events.

- ⇒ **Data integrity Enforcement:** Validation and confirm the user inputs, enforce referential integrity constraints and data type.
- ⇒ **Payout Functionality:** Transparent payment overview.
- ⇒ **Property Functionality:** Detail description of property related data.

Entity Relationship Model (ERM) for Airbnb Database:



DATA DICTIONARY:

1. User:

id(PK)	Unique identifier for each user.	int auto_increment
name	Full name of the user.	varchar(255) not null
email	Email address of the user.	varchar(255) unique not null
phone	Contact number of the user.	varchar(15)
user_type	Role of the user(Host, Guest, Admin)	enum(host, guest, admin) not null

2. Host:

id(PK)	Unique identifier for each host.	int auto_increment
user_id(FK)	References the id in the user table.	int not null
Bank_account_details	Details of bank account for payments.	Varchar(255) not null

3. Guest:

id(PK)	Unique identifier for each guest.	int auto_increment
user_id(FK)	References the id in the user table.	int not null
Payment_method	Preferred payment method of the guest.	Varchar(255) not null

4. Location:

id(PK)	Unique identifier for each location.	int auto_increment
country	Country where property locate.	varchar(255) not null
city	City where property locate.	varchar(255) not null
street	Street where property locate.	varchar(255) not null
post_code	Postal code where property locate.	Varchar(255) not null

5. Property:

id(PK)	Unique identifier for each property.	int auto_increment
host_id(FK)	References the id in the host table.	int not null
name	Name of the property.	varchar(255) not null
description	Details description of the property.	text
location_id(FK)	References the id in the location table.	int not null
price_per_night	Cost per night for the property.	decimal(10, 2) not null

6. Booking:

id(PK)	Unique identifier for each booking.	int auto_increment
property_id(FK)	References the id for the property table.	int not null
guest_id(FK)	References the id for the guest table.	int not null
check_in_date	Check in date for the booking.	date not null
check_out_date	Check out date for the booking.	date not null
total_price	Total price of the booking.	decimal(10, 2) not null
status	status of booking(pending, confirmed, cancelled, completed).	enum(pending, confirmed, cancelled, completed) not null

7. Payment:

id(PK)	Unique identifier for each payment.	int auto_increment
booking_id(FK)	References the id for the booking table.	int not null
amount	Total amount paid.	decimal(10, 2) not null
payment_date	Date of payment.	date not null
payment_method	Method used for payment.	varchar(255) not null

8. Review:

id(PK)	Unique identifier for each review.	int auto_increment
booking_id(FK)	References the id for the booking table.	int not null
reviewer_id(FK)	References the id for the user table.	int not null
Reviewee_id(FK)	References the id for the user table.	int not null
rating	Rating given (1 to 5).	int check(rating between 1 and 5)
comment	Review comment.	text
review_date	Date of the review.	date not null

9. Amenity:

id(PK)	Unique identifier for an amenity.	int auto_increment
name	Name of the amenity.	varchar(255) not null

10. PropertyAmenity:

property_id (PK), (FK)	Unique identifier of a property, References the id for the property table	int auto_increment, int not null
amenity_id(PK), (FK)	Unique identifier of an amenity, References the id for the amenity table.	int auto_increment, int not null

11. Availability:

id(PK)	Unique identifier for each availability.	int auto_increment
property_id (FK)	References the id for the property table.	int not null
date	Date of availability.	date not null
is_available	Boolean indicating if the property is available (True or False).	boolean not null

12. Commission:

id(PK)	Unique identifier for each commission.	int auto_increment
booking_id(FK)	References the id for the booking table that generated the commission.	int not null
commission_rate	The rate at which the commission is calculated.	decimal(10, 2) not null
commission_amount	The actual commission amount earned.	decimal(10, 2) not null

13. Payout:

id(PK)	Unique identifier for each payout.	int auto_increment
host_id(FK)	Reference the id for the host table receiving the payout.	int not null
amount	The amount paid out.	decimal(10, 2) not null
payout_date	The date the payout was made.	date not null
approved_by(FK)	References the id to the admin table who approved the payout.	int

14. Message:

id(PK)	Unique identifier for each message.	int auto_increment
sender_id(FK)	References the id for the user table who sent the message.	int not null
receiver_id(FK)	References the id for the user table who received the message.	int not null
message_text	The context of the message.	text not null
timestamp	The date and time the message was sent.	datetime not null

15. SocialMedia:

id(PK)	Unique identifier for each social media record.	int auto_increment
user_id(FK)	References the id for the user table associated with the social media profile.	int not null
platform_name	The name of the social media platform.	varchar(255) not null
profile_link	The link to user's profile on the platform.	varchar(255) not null

16. Dispute:

id(PK)	Unique identifier for each dispute.	int auto_increment
booking_id(FK)	References the id for the booking table associated with the dispute.	int not null
complaint_text	The details of the complaint.	text not null
resolved_by(FK)	References the id for the admin table who resolved the dispute.	int
resolution_status	The current status of the dispute (open, resolved, closed).	enum(open, resolved, closed) not null

17. Admin:

id(PK)	Unique identifier for each admin.	int auto_increment
name	The name of the admin.	varchar(255) not null
email	The email address of the admin.	varchar(255) unique not null
user_id(FK)	References the id for the user table associated with the admin.	int not null

18. Notification:

id(PK)	Unique identifier for each notification.	int auto_increment
user_id(FK)	References the id for the user table receiving the notification.	int not null
message	The content of the notification.	text not null
timestamp	The date and time the notification was sent.	datetime not null

19. Income:

id(PK)	Unique identifier for each income record.	int auto_increment
user_id(FK)	References the id for the user table who earned the income.	int not null
source	The source of the income.	varchar(255) not null
amount	The amount of income earned.	decimal(10, 2) not null
income_date	The date the income was earned.	date not null

20. Language:

id(PK)	Unique identifier for each language.	int auto_increment
name	The name of the language.	varchar(255) not null

21. UserLanguage:

user_id(PK), (FK)	Unique identifier for each user, References the id for the user table associated with userlanguage.	int auto_increment, int not null
language_id(PK), (FK)	Unique identifier for each language, References the id for the language table associated with user language.	int auto_increment, int not null